用 Coding Agent 完成端到端需求

本教程基于 Cursor Learn 的《综合运用》内容整理，目标是给你一个可以直接套用的工作流：

- 修改前先探索
- 写代码前先规划
- 上线前先测试
- 合并前先评审
- 经验沉淀成规则

适用场景：你接手一个不熟悉的代码库，需要在有限时间内交付一个完整功能。

教程目标

以“给结账流程添加折扣码”为例，完整走一遍协作闭环：

1. 理解代码库和数据流
2. 产出可执行计划
3. 实现最小可用版本（MVP）
4. 调试失败边界用例
5. 复查、测试、沉淀规则

步骤 1：先理解代码库（Explore）

先让 agent 帮你建立系统心智图，不要直接改代码。

可直接使用的提示词：

```
我需要在结账流程中加入折扣码功能。在修改任何代码之前，
请先带我梳理当前的结账流程：涉及哪些组件、API 路由和服务？
请展示从购物车到支付确认的完整数据流。
```

继续追问关键空白点：

```
我们的定价计算在 src/services/PricingService.ts 中是如何工作的？
我们在哪里计算订单总额？在这条流程里，折扣最适合在哪一步应用？
```

输出要求（建议你明确告诉 agent）：

- 关键文件列表（组件、API、服务）
- 从输入到支付确认的数据流
- 可以插入折扣逻辑的最小改动点

步骤 2：先规划再实现（Plan）

切到 Plan Mode（`Shift+Tab`），让 agent 先产出执行方案，再 Build。

建议的计划模板（`discount-codes.md`）：

```
# 折扣码

## 概述
为结账流程添加折扣码支持。用户输入折扣码，系统校验后在税前应用折扣。

## 实现方案
- 添加 DiscountService，并实现校验逻辑（过期、次数、可用性）
- 修改 PricingService.calculateTotal()，支持可选折扣参数
- 在 OrderSummary 增加折扣码输入和反馈
- 添加 /api/discount/validate 接口

## 任务
- [ ] 创建 discount_codes 表迁移
- [ ] 创建 DiscountService（含 applyDiscount）
- [ ] 在 PricingService.calculateTotal() 接入折扣
- [ ] 在 OrderSummary 添加折扣输入 UI
- [ ] 添加 /api/discount/validate 路由
- [ ] 为折扣类型和边界场景补充测试
```

规划原则：

- 第一版只做核心链路（输入 -> 校验 -> 应用 -> 展示）
- 去掉“本周不影响上线”的扩展项
- 先保证正确性，再追求完整性

步骤 3：构建 MVP（Build）

在计划确认后执行 Build，要求 agent 分批提交改动，而不是一次性“大改”。

建议你给 agent 的约束：

- 每一步说明改了什么、为什么改
- 优先复用现有模式，不引入不必要抽象
- 关键逻辑必须附带测试

MVP 完成标准（Definition of Done）：

- 用户可输入折扣码
- 后端可校验折扣码有效性
- 总价计算正确反映折扣
- 订单汇总可见折扣金额与最终金额

步骤 4：调试失败边界用例（Debug）

常见问题：百分比折扣 + 固定金额折扣叠加时，不同输入顺序得到不同总价。

排查提示词：

```
折扣叠加测试失败了。相同的一组折扣会因为应用顺序不同而得到不同总价。
请找出原因，并修复 src/services/DiscountService.ts 中折扣应用顺序的问题。
请展示每一步计算的前后数值（before/after）。
```

调试重点：

- 先复现，再定位，不要只改表面症状
- 打印每一步中间值（before/after）
- 固化稳定顺序（例如：先百分比，再固定金额）
- 增加回归测试，覆盖顺序无关性

步骤 5：复查与测试（Review & Test）

在推送前做一次“审稿人视角”检查：

```
请审查我为折扣码功能所做的全部改动。
检查是否存在 bug、缺失的错误处理、边界情况遗漏，以及是否与现有模式保持一致。
如果你是代码评审者，会重点指出哪些问题？
```

建议检查清单：

- 错误处理：无效码、过期码、并发使用冲突
- 边界值：超大折扣、负值、金额下限（不小于 0）
- 一致性：命名、分层、接口风格、日志与监控
- 测试性：单测/集成测试覆盖关键路径和异常路径

步骤 6：沉淀规则（Rule）

把本次经验写成规则，避免同类问题重复出现。

示例规则：

```
# 定价规则

- 先应用百分比折扣，再应用固定金额折扣
- 订单总额不得低于 $0
- 参考 src/services/PricingService.ts 的标准计算模式
```

规则落地建议：

- 放到项目规则文件（如 `AGENTS.md` 或 `.cursor/rules`）
- 代码评审时将其作为必检项
- 新人 onboarding 时纳入开发手册

一次可复用的最小流程

你可以把下面这段当作每个新需求的通用脚本：

1. `Explore`：先让 agent 解释现状和数据流
2. `Plan`：产出任务清单，删减到 MVP
3. `Build`：分批实现并保持可测
4. `Debug`：失败用例查根因并补回归测试
5. `Review`：用审稿标准自查
6. `Rule`：把经验沉淀成项目规则

这套流程的价值不是“更快写代码”，而是更稳定地交付可上线的结果。

参考链接

- Cursor Learn《综合运用》：https://cursor.com/cn/learn/putting-it-together